

Dalil

a buslahub product · dalil.buslahub.com

Product Requirements Document

Full-Stack SaaS Platform for Creative Agencies

Version	1.6 — RBAC Authorization Model
Status	Planning Phase
Date	June 2026
Audience	Founders, Engineering, Design, Investors
Business model	Subscription SaaS (multi-tenant)
Primary market	Branding, Web, Marketing Agencies & Freelancers

1. Executive Summary

Dalil is a multi-tenant subscription SaaS platform built to serve creative agencies — branding, web, marketing, packaging, and any domain that follows the same project delivery flow. It consolidates the entire agency lifecycle in a single product: from the first contact with a lead, through a fully customisable CRM pipeline, to a catalog of sellable services and packages, quotation and proposal with flexible milestone billing, blueprint-driven project execution, client approval via a dedicated portal, and fully automated invoicing.

Today's agencies stitch together three to five disconnected tools (CRM, project manager, file storage, client portal, invoicing) to deliver a single project. Dalil replaces that patchwork with one cohesive, configurable workflow under one subscription.

2. Problem Statement

2.1 Current Pain Points

- Agencies lose leads because CRM is separate from project tools — no single view from contact to contract.
- Sales pipelines are rigid; teams can't configure stages to match their actual sales process.
- Proposals are assembled manually with no version control, no amendment loop, and no client e-signature.
- Project structure is re-created from scratch on every engagement, wasting hours on setup.
- Clients give feedback via email or WhatsApp, breaking the paper trail and delaying approvals.
- Invoices are triggered manually, leading to missed billing and inconsistent cash flow.
- No unified audit log, so milestone approvals are undocumented and disputed.

2.2 Opportunity

The global creative-agency software market is fragmented. There is no dominant all-in-one platform that combines a configurable CRM + blueprint-driven project delivery + client portal + automated invoicing in one subscription product. Dalil targets that gap.

3. Product Vision & Goals

3.1 Vision Statement

“The operating system every creative agency needs — from first call to final invoice, in one place.”

3.2 Business Goals

- 1. Reach 200 paying agency tenants within 12 months of launch.
- 2. Reduce average agency onboarding time (first project live) to under 30 minutes.
- 3. Achieve < 5% monthly churn by ensuring clients renew because the product owns their workflow.
- 4. Expand service-type coverage to at least 10 domains without code changes (config-driven blueprints).

3.3 Success Metrics

Metric	Target	Measurement
Monthly Recurring Revenue (MRR)	SAR 185k by month 12	Stripe dashboard
Tenant activation rate	>80% create first project	Event analytics
Client portal engagement	>70% clients log in	Session tracking
Invoice automation rate	>90% invoices auto-triggered	BullMQ job logs
NPS	>50	In-app survey

4. Users, Roles & Personas

Dalil has three distinct actor types. Two are kept deliberately simple and separate; the third (the tenant user) draws all of its capability from the RBAC authorization model defined in Section 4A.

Actor	Level	Who	How access is granted
Platform Owner	Platform	The company operating Dalil	Hardcoded platform role. Manages plans, pricing, entitlements, and tenant lifecycle — never tenant operational data. Not part of tenant RBAC.
Tenant User	Tenant	Any authenticated person inside an agency (owner, sales rep, PM, designer, finance, etc.)	A single authenticated user type with no fixed role. All capability comes from membership in one or more Permission Groups (see Section 4A).
Client	Tenant (portal)	End customer of the agency	Completely separate from tenant RBAC: own authentication (passwordless), own portal, own API surface. Client actions are portal authentication, not tenant permissions.

Three concepts must never be conflated: a Persona is a human archetype used for illustration; a Permission Group is a named bundle of access that actually controls what a user can do; a Permission is the atomic authorization key that enforcement checks against. Personas illustrate, Permission Groups control, Permissions enforce.

4.1 Persona Profiles (Illustrative)

These personas are typical humans an agency employs. Each maps to one or more default Permission Groups — they are not system roles. A real tenant can split, merge, or rename the underlying groups freely.

Agency Administrator — “Sarah, Agency Director”

- Runs a 12-person branding + web agency; typically a member of the Administrator group.
- Pre-configures sales pipeline stages, blueprints, and task boards once — then the team executes repeatably.
- Manages team membership and Permission Groups; wants automated invoicing and audit trails.

Sales Representative — “Ali, Account Manager”

- Manages 20–40 leads through a pipeline the team defined; typically in the Sales group.
- Has CRM, contacts, and quotation permissions but not, by default, blueprint or invoicing permissions.
- Sends proposals and handles client amendment requests without leaving Dalil.

Project Manager — “Noura, Delivery Lead”

- Owns project execution; typically in the Project Manager group.
- Holds project, task-board, approval, and client-feedback permissions; starts projects and reviews team deliverables.
- Coordinates phases and unblocks the team across several active projects.

Designer — “Mia, Lead Designer”

- Works on 3–5 active projects; typically in the Designer group.
- Holds task, asset, and comment permissions scoped to assigned work; uploads deliverables and moves tasks to Review.
- Does not, by default, hold quotation, invoicing, or user-management permissions.

Finance Officer — “Omar, Accountant”

- Handles billing; typically in the Finance group.
- Holds invoice, payment-recording, and financial-analytics permissions; records partial payments and approves change-request invoices.
- Has no need for task-board or blueprint permissions and is not granted them by default.

Client — “Khaled, Marketing Director”

- Outside tenant RBAC entirely. Receives a one-time onboarding link to create his account, then uses passwordless re-entry.
- Reviews progress, views deliverables, comments, and approves milestones — all through Client Portal authentication, not tenant permissions.
- Can submit a new-project request; the agency still prepares a quotation and starts the project.

4A. Authorization Model (RBAC)

Dalil uses permission-based access control (RBAC) for tenant users, in the spirit of Google Workspace, Jira, or GitHub Enterprise. Tenant users have no hardcoded roles. Instead, permissions are the source of truth, permissions are bundled into Permission Groups, and users are assigned to groups. This lets an agency of any size model its own structure (Creative Director, QA, Accountant, Viewer, ...) without Dalil ever shipping a new hardcoded role.

4A.1 The Authorization Chain

Access resolves along a single chain: a User belongs to one or more Permission Groups (via `UserPermissionGroup`); each Permission Group holds a set of Permissions (via `PermissionGroupPermission`); each Permission is an atomic key such as `quotations.create`. A user's effective permissions are the union of all permissions across their groups (plus any optional per-user overrides). Every protected action checks for a specific permission key — never a role name.

4A.2 Permission Groups

- Each tenant owns its own Permission Groups; groups are tenant-scoped and never shared across tenants.
- A user with `permission-groups.manage` can create, rename, duplicate, delete, and configure the permissions of groups.
- Dalil ships sensible default groups (Section 4A.4) so a new tenant is productive immediately without configuring RBAC.
- Default groups are starting templates: tenants may edit, duplicate, or replace them. System-protected groups (Administrator) are flagged `isSystem` and cannot be deleted.

4A.3 Tenant Bootstrap

When a tenant is provisioned, RBAC is set up automatically so there is always a way in:

1. Create the Tenant.
2. Create the first Tenant User (the person who signed the agency up).
3. Create the tenant's default Permission Groups.
4. Assign the first user to the Administrator group.

The first user is not a special role — they are simply a member of the default Administrator Permission Group. From there they can invite others and shape the tenant's groups however they wish.

4A.4 Default Permission Groups (Starting Templates)

These ship with every tenant and can be customized. Each is a bundle of permissions from the Permission Catalog (Section 4B).

Default Group	Description	Default permission bundle
Administrator	Full tenant access; manages users, groups, and settings	All tenant permissions (system group, cannot be deleted)
Sales	Lead-to-proposal work	crm.*, contacts.*, quotations.*, plus projects.view
Operations	Day-to-day delivery setup	services.*, packages.manage, blueprints.*, projects.view, tasks.*
Project Manager	Owns project execution	projects.*, tasks.*, approvals.review, plus contacts.view and analytics.view
Finance	Billing and revenue	invoices.*, analytics.view, projects.view
Designer	Executes assigned creative work	tasks.view, tasks.manage (assigned), assets, comments, projects.view
Developer	Executes assigned build work	tasks.view, tasks.manage (assigned), assets, comments, projects.view
Viewer	Read-only where permitted	*.view permissions only (crm.view, projects.view, invoices.view, analytics.view, ...)

4A.5 Optional Per-User Overrides

The architecture supports granting or revoking an individual permission on a single user without creating a new group — for example, temporarily granting invoices.view to one designer. This is captured by the optional UserPermissionOverride entity. It is supported by the data model and resolution logic from day one but need not be exposed in the initial UI; group membership covers the common cases.

4A.6 Administrator Protection Rules (System Invariants)

These are platform safeguards, not business rules. They prevent a tenant from locking itself out and are enforced by the system regardless of how groups are configured.

- The Administrator Permission Group cannot be deleted (isSystem = true).
- A tenant must always have at least one user in the Administrator group.

- The last Administrator cannot remove themselves from the Administrator group.
- The last Administrator cannot be deactivated.
- The last Administrator cannot lose critical permissions — specifically users.manage, permission-groups.manage, and settings.manage.

4B. Permission Catalog

Permissions are grouped into categories and are the source of truth for authorization throughout Dalil. The catalog below is representative, not exhaustive — new permission keys may be added as features grow, but the naming convention (category.action) is consistent everywhere. Platform-level permissions are exclusive to the Platform Owner and are not part of this tenant catalog.

Category	Permission keys
CRM	crm.view, crm.create, crm.edit, crm.delete
Contacts	contacts.view, contacts.manage
Services	services.view, services.manage
Packages	packages.manage
Blueprints	blueprints.view, blueprints.manage
Projects	projects.view, projects.start, projects.archive
Tasks	tasks.view, tasks.manage, tasks.assign
Approvals	approvals.review
Quotations	quotations.view, quotations.create, quotations.send
Invoices	invoices.view, invoices.create, invoices.record_payment
Change Requests	change_requests.manage, change_requests.approve
Analytics	analytics.view
Users	users.view, users.invite, users.manage
Permission Groups	permission-groups.manage
Settings	settings.manage
Notifications	notifications.manage

Client portal actions (review, comment, approve milestone) are deliberately absent from this catalog: they are governed by Client Portal authentication, a separate mechanism from tenant RBAC.

5. System Architecture Overview

5.1 Technology Stack

Layer	Technology	Purpose
Backend framework	NestJS (Node.js)	Modular API server with DI, guards, interceptors
Real-time	Socket.io via @nestjs/websockets	Live task updates, notifications, portal sync
Database	PostgreSQL + Prisma ORM	Primary data store with type-safe migrations
Cache / Queue broker	Redis	Session cache, pub/sub, BullMQ backing store
Async job queue	BullMQ (@nestjs/bullmq)	Invoice triggers, email dispatch, automations
API validation	OpenAPI + Zod (nestjs-zod)	Schema-first request/response validation
File storage	AWS S3 + presigned URLs	Brand assets, deliverable uploads
Frontend framework	Next.js (App Router)	SSR + CSR hybrid, supports portal & admin
UI components	TailwindCSS + Shadcn UI	Consistent design system
Animation	GSAP + Framer Motion	Transitions, micro-interactions
i18n	i18next	Arabic (IBM Plex Sans Arabic) + English (Saudi Air)
State management	Zustand + TanStack Query	Local state + server state with socket sync
Forms	React Hook Form + @hookform/resolvers/zod	Validated, typed forms
E2E testing	Playwright	Critical path automation
Unit testing	Jest	Service & utility coverage
Auth	JWT + @UseGuards()	Stateless auth; multi-tenant + permission guards (checks permission keys, not roles)
CI/CD	GitHub Actions + Devcontainer	Automated test, build, deploy pipeline
Version control	GitHub	Source of truth

5.2 Pre-Configured Items (Admin Setup)

Before any project can run, an authorized user configures a set of foundational items. These are set up once and reused across all projects and clients on the tenant. They divide into two layers: the commercial layer (what the agency sells) and the operational layer (how the agency delivers).

Commercial Layer — What the Agency Sells

Item	What it defines	Reusability
Service	A sellable offering (e.g. Brand Identity, Web Development) with name, description, and default price. The atomic commercial unit.	Reused across packages, blueprints, and quotations
Package	A commercial bundle of multiple services sold at a combined price (e.g. 'Brand + Web Starter'). Expands to its service set at quotation time.	Reused across quotations

Operational Layer — How the Agency Delivers

Item	What it defines	Reusability
Custom Sales Flow	The stages of the lead pipeline (any number, any name, any order). Default example: Follow-Up → Prospect → Negotiation.	Applies to all leads on the tenant
Blueprint	The project structure: phases, each delivering one service; phase dependencies (parallel or sequential); and each phase's task board assignment. A blueprint covers an exact set of services.	Reusable across unlimited projects and clients
Task Board	A kanban board assigned to one or more phases within a blueprint. A phase can have a dedicated board or share a board with any other phase(s), regardless of service.	Defined at blueprint level; inherited by all projects using that blueprint

Setup Order (Required Sequence)

- Services must be created first — they are referenced everywhere downstream.
- Blueprints are built next; each phase selects a service (which must already exist) and its dependencies.

- Packages can be created any time after services exist; a package simply groups existing services.
- Custom sales flow and task boards are configured independently at any point.

5.3 Multi-Tenancy Model

- Each agency is an isolated tenant identified by a unique tenant ID.
- All database queries are scoped with tenant ID at the ORM level via Prisma middleware.
- Deep Route Guards (@UseGuards) enforce tenant isolation on every API endpoint.
- Client portal URLs are tenant-scoped (e.g., /portal/:tenantSlug/:projectId).
- File storage uses tenant-prefixed S3 paths to prevent cross-tenant access.

5.4 Security & Compliance

- Immutable audit logs track every User ID milestone approval with timestamp and IP.
- Presigned S3 URLs expire after a configurable TTL; no direct bucket access.
- Permission-based access control (RBAC) enforced at the controller and service layers: every protected endpoint declares a required permission key (e.g. quotations.create) and is checked via @UseGuards against the user's effective permissions — never against a role name.
- All API inputs validated with Zod schemas before reaching business logic.
- Core error screens: 404 (Not Found), 403 (Forbidden), 500 (Global Error Boundary).

6. Happy Flow — End-to-End Workflow

The platform workflow is divided into four macro-flows: CRM & Lead Management, Quotation & Proposal, Project Blueprint Configuration, and Project Execution. All four macro-flows depend on three pre-configured items set up by authorized users before any project begins (see Section 5.2).

6.1 Macro-Flow A — CRM & Lead Management

Step 1 — New Lead Entry

- A contact enters the system when a user sends a message or books a call.
- The Sales team is notified and assigned to follow up.
- The lead is placed into the tenant's Custom Sales Flow — a pipeline of stages configured by a user with `crm.edit` before any leads arrive.
- The number of stages, their names, and their order are fully flexible. A user with the `crm.edit` permission can define as few as one stage or as many as needed (e.g. Follow-Up → Prospect → Negotiation, or a simpler or more complex flow).
- Sales team members move the lead through stages manually as the relationship progresses.

Step 2 — Conversion Decision

- At any stage, the Sales team records a Conversion Status decision.
 - YES → the system generates a magic link and sends it to the lead.
 - The lead clicks the magic link, creates their account, and their Client Portal is provisioned automatically.
 - NO → the lead is marked “Stopped” and archived. The sales cycle ends here.

Step 3 — Contact Creates a Project

- Once converted, a Contact record can spawn one or more Projects over their lifetime.
- The Admin initiates the project. The client may also submit a new-project request from within their portal (a request, not a self-served project — it still requires an admin quotation and Start; see 7.7).

Step 4 — Returning-Client Re-Entry (Portal Login)

- The magic link in Step 2 is a one-time onboarding link that creates the account. It is not how the client logs in again later.
- For every subsequent visit, the client uses passwordless re-entry: on the client login screen they enter their email and receive a short-TTL, single-use login link (15–30 minutes) that signs them in.

- Optionally, after first sign-in a client may set a password and use email + password on future visits. Passwordless remains available as a fallback for password-set clients (e.g. forgotten password).
- Onboarding links and login links are distinct token types with different TTLs (onboarding 7 days, login 15–30 minutes) and are tracked separately.
- Because client identity is keyed by email across tenants, the same login resolves to whichever tenant portal(s) the client belongs to.

6.2 Macro-Flow B — Quotation & Proposal

Step 1 — Admin Creates a Quotation

- A user with the quotations.create permission opens the Quotation Builder and selects the Contact.
- They select the services to be delivered — either by picking individual Services or by picking a Package (which expands to its underlying services).
- The system filters Blueprints by exact service-set match: only blueprints whose service set is exactly equal to the selected services are shown.
 - Example: selecting {Web Development} shows only blueprints covering exactly {Web Development} — not {Web Development, Brand} and not a subset.
 - Example: selecting {A, B} shows only blueprints covering exactly {A, B} — never {A} alone or {A, B, C}.
 - If no blueprint matches the selected services exactly, the system shows 'No blueprint found — create one' and links to the blueprint editor.
- They pick one matching blueprint, which determines the project phases and their structure.
- Quotation total is computed as follows: when individual services are selected, the total defaults to the sum of each selected service's default price; when a package is selected, the total defaults to the package's bundled price (the package price is authoritative and is not re-derived from its services). The total can be overridden with a manual figure on the quotation. A package's bundled price does not auto-decompose per service or per phase — milestone amounts are assigned manually (see milestone grouping below), and those amounts must sum to the total.
- They define the Payment Model. This must be fully configured before the proposal is sent or the project starts:
 - One-Time Payment: a single billing checkpoint covering all phases. Timing is configurable — collected before project start, or after the final phase completes.
 - Milestone-Based: phases are grouped into one or more milestones. Each milestone triggers a single invoice when all of its phases are completed and client-approved.
 - Milestone grouping: billing boundaries are drawn across the blueprint's phases. Each phase belongs to exactly one milestone. The rule that replaces simple contiguity (which doesn't hold once phases run in parallel): a milestone must be dependency-closed — if two phases are in the same milestone and a third phase lies on a dependency path between them, that third phase must also be in the milestone. In plain terms, a phase cannot sit in a later milestone than something that depends on it. Parallel phases (no dependency path between them) may be grouped together or separately.

- Milestone firing order: a milestone's invoice cannot fire until every milestone it depends on (derived from phase-level dependencies) has fired — so a fast parallel branch never bills ahead of the earlier work it logically follows.
- Every phase must belong to exactly one milestone; milestone amounts must sum to the quotation total.
- Under the hood, one-time payment is simply a single milestone containing all phases — unifying both models under one mechanism.

Step 2 — Proposal Sent to Client

- The system generates a branded Proposal document from the quotation.
- A user with the quotations.send permission sends the Proposal to the client via email; it also appears in their portal.

Step 3 — Client Reviews the Proposal

- The client opens the Proposal in their portal.
- Client can comment on the proposal and request amendments.
- The amendment request is received by an authorized user, who updates the quotation and resends the revised proposal.
- This amendment loop can repeat until both parties are satisfied.

Step 4 — Client Accepts, Project Starts

- Client clicks Accept on the final proposal version.
- A user with the projects.start permission receives an acceptance notification and reviews the project setup.
- That user clicks Start Project — this is the gate that makes the project active.
- Only after Start is clicked can the client track project progress in the portal.

6.3 Macro-Flow C — Project Blueprint & Board Configuration

Blueprints and task boards are pre-configured (by users with blueprints.manage) before any project starts. When a quotation is created, an existing blueprint is selected — phases are not configured per-project.

Blueprint Structure

- A blueprint is created and owned at the tenant level by a user with blueprints.manage. Services must already exist before a blueprint can be built.
- A blueprint can contain one phase or many phases — the number is fully flexible.
- Each phase has its own name and delivers exactly one service (selected from the pre-configured services).

- A single service can be delivered across multiple phases. Example: the Web Development service split into 'Major features' and 'Minor features' phases, purely so each becomes its own billable milestone.
- Phase name and phase service are distinct: two phases can share the same service but have different names.
- The blueprint's service set is the union of all its phases' services — this is what the exact-match filter uses at quotation time.
- Phase dependencies are configured inside the blueprint:
 - Sequential: Phase B cannot start until Phase A is completed.
 - Parallel: Phase B and Phase C can run simultaneously.
- The same blueprint can be linked to multiple projects and multiple clients simultaneously.

Task Board Configuration (Inside Blueprint)

- For each phase, the admin makes an explicit board choice:
 - Create a new dedicated board for this phase, or
 - Share an existing board with any other phase(s) in the blueprint.
- Board sharing is independent of service: phases sharing a board may deliver the same service or different services — entirely the admin's choice.
- The board assignment is fixed at blueprint level and inherited by all projects using that blueprint.
- Task templates are pre-populated into the board columns on project initialization.

Shared Board Rules (Ownership & Progress)

- A task belongs to exactly one phase. Every task carries a single `projectPhaseId`; the board is only a view, never an owner. A task can never belong to multiple phases — work spanning phases is modeled as separate tasks.
- There is no stored 'board progress'. Progress is always computed per phase: a phase's progress = its completed tasks ÷ its total tasks. A shared board simply renders tasks from several phases, each contributing to its own phase's progress.
- A phase is marked complete only when all of its tasks are done AND it has passed client approval — approval is the gate, not the task count alone.
- A shared board provides a phase filter (all phases / a single phase) and shows a phase badge on every card, so the team always knows which phase a task belongs to and can compute per-phase completion by filtering first.

Assets

- Brand assets (logos, fonts, colour palettes) are uploaded to S3 and linked to the relevant phase (typically Phase 1 — Brand).

6.4 Macro-Flow D — Project Execution (Phase 1: Kickoff)

Three-Lane Board (Team | Admin | Client)

Team Lane

- Team members see the Task Board assigned to the active phase.
- A 'New Concept' flag appears when a concept card is added and not yet seen by Admin.
- Tasks progress through board columns (To-Do → In Progress → Review → Done).

Reviewer Lane (approvals.review)

- A user with the approvals.review permission reviews completed work on the board.
- Triggers the Project Owner Approval decision:
 - NO → the reviewer adds a comment; work returns to the Team Lane with feedback.
 - YES → deliverables are pushed to the Client Portal.

Client Lane

- Client receives a portal notification and opens the deliverables for review.
- Client can:
 - Give Feedback → comment is recorded and looped back to the Team (feedback loop).
 - Approve → the phase is marked Phase Completed.

Phase Completion & Payment Automation

- When a phase reaches 100% client approval, the automation checks whether it is the last unfinished phase in its milestone:
 - If other phases in the same milestone are still open → no invoice yet; wait.
 - If it is the final phase of the milestone → BullMQ job fires → invoice generated for that milestone → sent via email and portal notification.
 - One-time (before start): the single all-phase milestone is triggered at project start instead of on completion; invoice is collected upfront.
 - One-time (after finish): the single all-phase milestone fires only when the final phase of the project completes.
- Overtime payment alerts fire if a phase exceeds its ProjectPhase.deadline (materialized at Start from the blueprint phase's optional duration).
- Next sequential phase unlocks automatically; parallel phases are already running.

Progress Dashboard

- A project-level progress bar shows overall % complete.
- Each phase has its own progress bar.

- Phase statuses: Completed | Active | Pending (unlocks per dependency rules).

7. Feature Modules

7.1 Services & Packages Module

Feature	Description	Priority
Services catalog	Users with services.manage create sellable services (name, description, default price). Atomic commercial unit reused everywhere.	P0
Service ↔ blueprint linkage	A service can appear in multiple blueprints; many-to-many. Linkage derived from blueprint phases.	P0
Packages / bundles	Users with packages.manage group multiple services at a combined price; expands to its service set at quotation time.	P1
Default pricing	Each service carries a default price; editable per quotation.	P0
Service usage view	See which blueprints and packages reference a given service before editing/deleting it.	P2

7.2 CRM Module

Feature	Description	Priority
Lead capture	Manual entry or future webhook from website forms	P0
Custom sales flow config	Users with crm.edit define pipeline stages: any number, any name, any order. Default example: Follow-Up → Prospect → Negotiation	P0
Pipeline view	Kanban board reflecting the tenant's configured stages + Converted + Archived	P0
Lead stage movement	Sales team drags or updates lead stage manually	P0
Contact profile	Full history: messages, calls, proposals, projects	P0

Conversion action — magic link	On YES: system generates and sends magic link to lead; lead creates account; portal provisioned automatically	P0
Activity log	Timestamped log of all sales actions per lead	P1
Bulk actions	Mass status update, assign, archive	P2

7.3 Quotation & Proposal Module

Feature	Description	Priority
Quotation builder	Select contact, select services or a package, filter blueprints by exact service match, pick blueprint, define payment model	P0
Service / package selection	A user with quotations.create picks individual services or a package; package expands to its underlying service set	P0
Exact-match blueprint filter	Show only blueprints whose service set exactly equals the selected services; no subset or superset	P0
No-match fallback	If no blueprint matches, show 'No blueprint found — create one' with link to blueprint editor	P0
Milestone grouping	Group blueprint phases into billing milestones; each milestone must be dependency-closed (no phase grouped later than something that depends on it); fires one invoice	P0
Milestone firing order	A milestone's invoice waits until all milestones it depends on have fired; prevents a parallel branch billing out of order	P0
Payment model — one-time	Single all-phase milestone; timing configurable (before start or after finish)	P0
Payment model — milestone-based	Multiple milestones; each auto-invoices when all its phases are client-approved	P0

Total computation	Total defaults to sum of selected service default prices, or to a package's bundled price; admin-overrideable. Package price does not auto-decompose per phase.	P0
Milestone amount validation	Milestone amounts must sum to quotation total; every phase covered exactly once	P0
Proposal generator	Auto-generate branded PDF proposal from quotation	P0
Send proposal	Email + portal delivery with read tracking	P0
Client amendment request	Client comments on proposal requesting changes; admin is notified	P0
Proposal revision loop	An authorized user revises the quotation, regenerates proposal, resends; loop repeats until accepted	P0
Accept flow	Client accepts → a user with projects.start is notified → they click Start → project goes live	P0
Reject flow	Client rejection moves the proposal to REJECTED. The admin can revise it, which creates a new version and moves it to REOPENED → SENT on resend. Proposal status enum: DRAFT, SENT, AMENDMENT_REQUESTED, ACCEPTED, REJECTED, REOPENED.	P1
Revision history	Track all versions of a proposal	P1

7.4 Blueprint & Board Configuration Module

Feature	Description	Priority
Blueprint library	Users with blueprints.manage create and manage reusable blueprint templates at tenant level	P0
Phase management	Add / remove / reorder phases; each phase has a name and delivers exactly one service	P0
Service per phase	Each phase selects one service	P0

	(must already exist); a service can span multiple phases	
Service-set derivation	Blueprint's service set = union of its phases' services; used for exact-match filtering	P0
Phase dependency config	Define per-phase: sequential (depends on another phase) or parallel (runs simultaneously)	P0
Task board assignment	Per phase: create dedicated board or share with any other phase(s), independent of service	P0
Task template injection	Pre-populate board columns with task templates on project initialization	P0
Blueprint reuse	One blueprint can be linked to unlimited projects across multiple clients	P0
Blueprint snapshot on Start	On project Start, phases are copied into ProjectPhase rows; later blueprint edits never mutate running projects. This is the versioning mechanism (no separate version entity).	P0

7.5 Task Board Module

Feature	Description	Priority
Kanban board	Configurable columns per board; drag-and-drop cards	P0
Dedicated vs shared board	Board can serve one phase or multiple phases from the same blueprint; task always owned by exactly one phase	P0
Phase badge on cards	Each card shows which phase it belongs to (essential on shared boards)	P0
Phase filter	Filter a shared board to all phases or a single phase; per-phase progress computed on the filtered set	P0
Task cards	Title, description, assignee, due date, attachments, status	P0
Concept flags	Visual badge on new concept	P0

	cards not yet seen by Admin	
Phase status indicator	Active / On Hold per phase with visual indicator	P0
File attachments	Upload deliverables (S3 presigned) directly from task card	P0
Activity feed	Per-task comment and status change history	P1
Real-time updates	Socket.io pushes board changes to all connected team members	P0

7.6 Admin Approval Module

Feature	Description	Priority
Approval queue	Consolidated view of deliverables pending Admin review	P0
Approve action	Pushes deliverables to client portal; creates immutable audit log entry	P0
Request changes action	Adds comment, returns task to team, logs reviewer ID + timestamp	P0
Immutable audit log	Every approval/rejection recorded with User ID, timestamp, IP — append-only, no delete API	P0

7.7 Client Portal Module

Feature	Description	Priority
Magic link onboarding	Lead receives one-time onboarding link → clicks → creates account → portal provisioned. No password friction at entry.	P0
Passwordless re-entry	Returning client enters email → receives short-TTL single-use login link (15–30 min) to sign back in. This is the primary day-to-day login path.	P0

Optional set-password	After first sign-in, client may set a password for email+password login; passwordless stays available as fallback.	P1
Project list	Client sees all their projects with overall progress indicators	P0
Project overview	Progress bars for overall project and each phase; phase status (Completed / Active / Pending)	P0
Proposal review & amendment	Client views proposal, comments with amendment requests, accepts or rejects	P0
Project tracking (post-start)	Client can track project progress only after a user with projects.start clicks Start	P0
Deliverable viewer	Inline preview of images, PDFs, videos without download	P0
Feedback tool	Annotate / comment on specific deliverables; feedback loops back to team	P0
Approve milestone	One-click approval triggers automation chain (invoice if milestone-based)	P0
Comment history	Full thread of client ↔ team comments per deliverable	P1
New project request	Client submits a new-project request from the portal. It is a request only — the admin must create a quotation and click Start before it becomes an active project.	P1
Invoice view	Client can view and download invoices from portal	P1
Asset uploads	Client can upload brand assets (brief, logos, references) from portal	P1

7.8 Invoicing & Payment Automation Module

Feature	Description	Priority
Milestone completion trigger	BullMQ job fires when the last phase in a milestone reaches 100% client approval	P0

One-time before-start collection	Single all-phase milestone triggered at project start (when a user with projects.start clicks Start)	P0
One-time after-finish trigger	Single all-phase milestone fires when the project's final phase completes	P0
Invoice generation	Creates invoice from a milestone OR a change request; includes line items and due date	P0
Email delivery	Sends invoice to client email	P0
Portal notification	In-portal notification + invoice view updated on send	P0
Partial payments	Invoice tracks amountPaid vs amount; status derives as UNPAID / PARTIALLY_PAID / PAID / OVERDUE. Admin records payments manually; each is logged in a Payment row.	P1
Payment history	Per-invoice list of recorded payments (amount, date, method) for audit	P1
Invoice PDF export	Branded PDF download for client and agency records	P0

7.9 Change Requests & Additional Charges Module

Real engagements change scope (“client requested 10 extra pages”). A change request is a lightweight, admin-controlled adjustment that produces its own standalone invoice, outside the milestone structure. Overtime charges are modeled as a type of change request, unifying both into one mechanism.

Feature	Description	Priority
Create change request	A user with change_requests.manage creates a change request on a project (description + amount). A client may submit one as a request from the portal.	P1
Approval	A user with change_requests.approve approves or rejects; approval is required before any invoice is generated. Logged in the audit trail.	P1

Additional invoice	An approved change request generates a standalone invoice (links via <code>Invoice.changeRequestId</code>), separate from milestone invoices.	P1
Overtime as change request	A phase-deadline overage is entered as a change request of type OVERTIME — same approve-then-invoice path.	P1
Status tracking	Change request status: REQUESTED / APPROVED / REJECTED / INVOICED	P1

7.10 Notification & Automation Module

Feature	Description	Priority
In-app notifications	Real-time via Socket.io for all role-relevant events	P0
Email notifications	BullMQ queued; templated per event type	P0
Portal notifications	Client portal shows unread badge for new deliverables, invoice, proposal updates	P0
Automation rules	Users with <code>settings.manage</code> configure trigger → action rules (phase complete → invoice, etc.)	P0
Overtime payment alert	Auto-alert when a phase exceeds its <code>ProjectPhase.deadline</code>	P1

7.11 Team & Access Management Module

Covers inviting tenant users, assigning them to Permission Groups, and workload visibility. Launch scope for analytics-style team features is deliberately minimal (workload only); capacity planning and time tracking are deferred (see 7.11.1).

Feature	Description	Priority
Invite & manage users	Users with <code>users.invite</code> invite tenant users by email; <code>users.manage</code> edits or deactivates them	P0
Permission Group management	Users with <code>permission-</code>	P0

	groups.manage create, rename, duplicate, delete (except system Administrator), and configure group permissions	
Assign groups to user	Users with users.manage assign or remove a user's Permission Groups; optional per-user permission override	P0
Workload view	Per-member list and count of active assigned tasks across all projects — answers 'who is overloaded?'. Derived from existing Task data; no new entity.	P1
Assignment from task card	Assign / reassign a task to a team member (requires tasks.assign)	P0

7.11.1 Deferred (Specified, Not Built in v1)

- Capacity planning: define each member's availability/hours and compute utilization against it. Deferred until tenant volume justifies planning.
- Time tracking: logging hours per task (timers, manual entry, reporting). Held entirely for v1; planned as a fast-follow module.

7.12 Analytics & Reporting Module

Analytics ships as a small set of at-a-glance numbers (visible to users with analytics.view), computed on read from existing data. Metrics that require accumulated history (trends, velocity, bottleneck analysis) are deferred — but the transition-timestamp logging that feeds them is captured from day one (see 7.12.2), because that history cannot be reconstructed later.

7.12.1 Built in v1 (computed on read)

Area	Metric	Derivation
Sales	Lead conversion rate	converted leads ÷ total leads
Sales	Proposal win rate	accepted proposals ÷ sent proposals
Delivery	Overdue projects	count of phases past ProjectPhase.deadline
Delivery	Active projects by status	count grouped by project status
Finance	Revenue	sum of paid invoice amounts
Finance	Outstanding invoices	sum of unpaid + partially-paid balances

Finance (platform)	MRR per tenant	tenant plan price × active subscriptions (Platform Owner)
--------------------	----------------	---

7.12.2 Foundation Captured Now, Surfaced Later

- Transition timestamps are logged from day one: every lead stage change and every phase status change writes a timestamped TransitionLog row, even though no analytics UI consumes them yet.
- This unlocks deferred analytics later without backfill: pipeline velocity (avg time per sales stage), project completion time, and phase bottleneck analysis (avg time-in-phase).
- Deferred analytics UI (velocity, completion time, bottlenecks, revenue trends over time) is specified but not built in v1.

8. Screen Inventory

The following 60 screens cover every state and transition in the happy flow, including returning-client portal re-entry. The Access column shows the permission key (or actor) that governs each screen; a screen is visible to any tenant user whose Permission Groups grant the listed permission.

8.1 Auth Screens

#	Screen	Access (permission / actor)	Key Elements
1	Login	Tenant User	Email + password, forgot password link
2	Forgot password / reset	Tenant User	Email entry, reset token, new password form
3	Magic link landing (onboarding)	Client	One-time onboarding token validation, account creation form, portal redirect on success
3a	Client login (re-entry)	Client	Email entry → passwordless login link sent (15–30 min, single-use); or email + password if the client has set one
3b	Client set-password (optional)	Client	Offered after first sign-in; lets the client set a password for future email+password login; passwordless remains a fallback

8.2 Services & Packages Screens

#	Screen	Access (permission / actor)	Key Elements
4	Services catalog	services.manage	List of services with default price, linked blueprint count; add / edit / delete
5	Service editor	services.manage	Name, description, default price; usage

			view (which blueprints/packages reference it)
6	Packages list	packages.manage	List of packages with included services and bundled price
7	Package editor	packages.manage	Name, select services to bundle, set combined price

8.3 CRM Screens

#	Screen	Access (permission / actor)	Key Elements
8	Leads pipeline board	crm.view	Kanban columns matching custom sales flow stages + Converted + Archived; lead cards with status badges
9	Lead detail / contact profile	crm.view, contacts.view	Contact info, activity log, linked proposals and projects, conversion action button
10	New lead form	crm.create	Contact fields, source, assign to sales member
11	Custom sales flow configurator	crm.edit	Add/remove/rename/reorder pipeline stages; drag-and-drop stage ordering

8.4 Quotation & Proposal Screens

#	Screen	Access (permission / actor)	Key Elements
12	Quotations list	quotations.view	All quotations with status, contact name, blueprint, payment model, created date
13	Quotation builder	quotations.create	Contact selector, service/package picker, exact-match blueprint

			filter, blueprint selector, payment model, total price
14	Milestone grouping step	quotations.create	Phases laid out in order; draw billing boundaries; name each milestone; assign amounts; live total validation
15	Proposal preview	quotations.view	Branded generated document; send button; version history
16	Proposal sent / tracking view	quotations.send	Read status, amendment requests received, resend option
17	Proposal amendment review	quotations.create	Client's comments, revise quotation action, regenerate & resend

8.5 Blueprint & Board Configuration Screens

#	Screen	Access (permission / actor)	Key Elements
18	Blueprint library	blueprints.view	List of blueprints with phase count, service set, linked project count
19	Blueprint editor	blueprints.manage	Add/remove/reorder phases; name each phase; select one service per phase; configure dependencies (sequential/parallel); assign board (new or shared)
20	Task board template editor	blueprints.manage	Define columns, add task templates per column, assign to phase(s)

8.6 Project Management Screens

#	Screen	Access (permission / actor)	Key Elements
---	--------	-----------------------------	--------------

21	Projects list / dashboard	projects.view	All active projects; filter by status, phase, blueprint; overall progress indicators
22	Project detail — overview	projects.view	Overall progress bar, phase list with individual progress bars and statuses, timeline, milestone summary
23	Project setup / start screen	projects.start	Review accepted proposal, confirm blueprint, confirm milestones, Start Project button
24	Phase detail	projects.view, tasks.view	Task board for this phase (or shared board), asset panel, phase progress, approval status

8.7 Task Board Screens

#	Screen	Access (permission / actor)	Key Elements
25	Kanban task board	tasks.view	Board columns; concept cards with New Concept flag; drag-and-drop; phase status badge (Active / On Hold)
26	Task card detail / modal	tasks.manage	Title, description, assignee, due date, attachments (S3 upload), comments, status selector, activity feed

8.8 Admin Approval Screens

#	Screen	Access (permission / actor)	Key Elements
27	Approval queue	approvals.review	Pending deliverables across all projects; sorted by project and phase

28	Approval review	approvals.review	Inline deliverable preview, comment field, Approve / Request Changes buttons, audit log entry confirmation
----	-----------------	------------------	--

8.9 Invoicing & Change Requests Screens

#	Screen	Access (permission / actor)	Key Elements
29	Invoices list	invoices.view	All invoices; derived status (Unpaid / Partially paid / Paid / Overdue); filter by project, source (milestone or change request)
30	Invoice detail / preview	invoices.view	Line items, source (milestone or change request), amount vs amountPaid, payment history, record-payment action, PDF export
30a	Record payment	invoices.record_payment	Enter amount, date, method; increments amountPaid and recomputes status
30b	Change request	change_requests.manage (client may request)	Description + amount; type SCOPE or OVERTIME; approve/reject; on approval generates an additional invoice

8.10 Team & Settings Screens

#	Screen	Access (permission / actor)	Key Elements
31	Team members list + invite	users.invite	Team roster, group memberships, invite by email, deactivate member
31a	Workload view	users.view	Per-member count and list of active assigned

			tasks across projects
31b	Assign groups to user	users.manage	Assign / remove a user's Permission Groups; optional per-user permission override
31c	Permission Groups	permission-groups.manage	List groups, create, duplicate, edit permissions (by category), delete (except system Administrator), assign users
32	Tenant settings	settings.manage	Agency name, logo, subscription plan, billing info
33	Notification preferences	Tenant User	Toggle email and in-app notification types per event
34	User profile & password	All authenticated	Name, email, password change, avatar

8.10b Analytics Screens

#	Screen	Access (permission / actor)	Key Elements
34a	Tenant analytics dashboard	analytics.view	At-a-glance: lead conversion rate, proposal win rate, overdue projects, active projects, revenue, outstanding invoices
34b	Platform analytics dashboard	Platform Owner	MRR per tenant, plan distribution, aggregate tenant health

8.11 Sales Screens

#	Screen	Access (permission / actor)	Key Elements
35	Sales dashboard	crm.view (scoped)	My leads summary, conversion rate, recent activity; scoped to

			assigned leads
--	--	--	----------------

8.12 Client Portal Screens

#	Screen	Access (permission / actor)	Key Elements
36	Portal home / project list	Client	All client's projects with overall progress; new project request button
37	Proposal review screen	Client	Proposal document view, comment / amendment request input, Accept / Reject buttons
38	Project 'not yet started' holding state	Client	Accepted proposal confirmation; message that project will begin shortly (shown before a user with projects.start clicks Start)
39	Project overview	Client	Overall progress bar, phase list with statuses (Completed / Active / Pending), milestone/payment summary
40	Phase deliverables view	Client	List of deliverables for the active phase; unread badge on new items
41	Deliverable viewer	Client	Inline preview (image / PDF / video), annotation / comment tool, feedback thread
42	Approval confirmation	Client	Approve milestone action → success state with next steps info
43	Invoice view	Client	Invoice list with status; PDF download per invoice

8.13 Platform Administration Screens (Platform Owner)

#	Screen	Access (permission /	Key Elements
---	--------	----------------------	--------------

		actor)	
44	Platform dashboard	Platform Owner	Tenant count, MRR, plan distribution, system health — aggregate metrics only
45	Tenants management	Platform Owner	List all tenants, subscription status, provision / suspend / reactivate; no access to tenant content
46	Plans & pricing editor	Platform Owner	Create/edit/retire plans, set price and currency, edit per-plan entitlement flags
47	Platform settings	Platform Owner	Global config, announcements, billing settings

8.14 System / Shared Screens

#	Screen	Access (permission / actor)	Key Elements
48	Notifications panel / drawer	All authenticated	Real-time notification feed; mark as read; link to relevant entity
49	404 — Not Found	All authenticated	Friendly error with navigation back to home
50	403 — Forbidden	All authenticated	Access denied message; missing-permission explanation
51	500 — Global error boundary	All authenticated	Generic error with retry and support contact

Total: 60 screens across 15 screen groups.

9. Data Model — High-Level Schema

All entities are scoped by tenantId. Below are the core entities and their primary relationships.

Entity	Key Fields	Relationships
Tenant	id, name, slug, planId, subscriptionStatus, createdAt	1–N Users, Projects, Blueprints, Services, SalesFlowStages; N–1 Plan
Plan	id, name, price, currency, billingPeriod, isActive	1–N Tenants; 1–N Entitlements (platform-managed)
Entitlement	id, planId, key, value	N–1 Plan (e.g. white_label=true, max_users=10, storage_gb=2)
PlatformUser	id, email, role (platform_owner), passwordHash	Platform-level; not scoped to any tenant
SalesFlowStage	id, tenantId, name, order	N–1 Tenant; 1–N Leads (via stage)
User	id, tenantId, name, email, passwordHash, isActive	N–1 Tenant; N–N PermissionGroups (via UserPermissionGroup); N–N Projects (via assignment). No role column — access is entirely via groups.
PermissionGroup	id, tenantId, name, description, isSystem, isEditable	N–1 Tenant; N–N Permissions (via PermissionGroupPermission); N–N Users
Permission	id, key, category, description	Global catalog (e.g. quotations.create); N–N PermissionGroups
PermissionGroupPermission	permissionGroupId, permissionId	Link table: which permissions a group grants
UserPermissionGroup	userId, permissionGroupId	Link table: which groups a user belongs to
UserPermissionOverride	id, userId, permissionId, effect (GRANT REVOKE)	Optional per-user override; supported by resolution logic, not necessarily in launch UI
Contact	id, tenantId, name, email, phone, passwordHash?, accountCreatedAt	1–N Leads, Projects, AuthTokens
AuthToken	id, contactId, type (ONBOARDING LOGIN), tokenHash, expiresAt, usedAt	N–1 Contact; onboarding TTL 7d, login TTL 15–30m, both single-use

Lead	id, tenantId, contactId, stageId, assignedTo, conversionStatus	N–1 Contact; N–1 SalesFlowStage
Service	id, tenantId, name, description, defaultPrice	N–N Blueprints (via phases); N–N Packages
Package	id, tenantId, name, bundledPrice	N–N Services (via PackageService)
PackageService	id, packageId, serviceId	Link table: Package ↔ Service
Blueprint	id, tenantId, name, description, createdAt	1–N Phases; N–N Projects; serviceSet derived from phases
Phase	id, blueprintId, name, serviceId, order, boardId, dependsOnPhaseIds[], defaultDurationDays?	N–1 Blueprint; N–1 Service; N–1 TaskBoard
TaskBoard	id, tenantId, name	1–N Phases (shared or dedicated); 1–N TaskColumns
TaskColumn	id, boardId, name, order	1–N Tasks
Quotation	id, tenantId, contactId, blueprintId, packageId?, paymentModel, oneTimeTiming?, totalAmount (defaulted from services/package, admin-overrideable), totalOverridden (bool), status	1–1 Proposal; 1–N Projects; 1–N QuotationMilestones; N–N Services (selected)
QuotationMilestone	id, quotationId, name, order, amount, triggerType, status	N–1 Quotation; 1–N MilestonePhases
MilestonePhase	id, milestoneId, phaseId	Link table: Milestone ↔ Phase (each phase in exactly one milestone)
Proposal	id, tenantId, quotationId, versionNo, status (DRAFT SENT AMENDMENT_REQUESTED ACCEPTED REJECTED REOPENED), sentAt, amendments[]	N–1 Quotation
Project	id, tenantId, contactId, quotationId, blueprintId (origin ref only), status, startedAt, progress	1–N ProjectPhases (snapshotted from blueprint at Start)
ProjectPhase	id, projectId, phaseId (origin ref), name, serviceId, boardId, dependsOnPhaseIds[], status, progress, deadline?, unlockedAt, completedAt	1–N Tasks; 1–N Assets — self-contained snapshot, independent of later blueprint edits

Task	id, projectPhaseId, columnId, title, status, assignedTo, dueDate, attachments[]	N–1 ProjectPhase
Asset	id, projectPhaseId, s3Key, type, uploadedBy, uploadedAt	N–1 ProjectPhase
Approval	id, projectPhaseId, reviewerId, decision, comment, timestamp, ipAddress	N–1 ProjectPhase; N–1 User (immutable)
ClientFeedback	id, projectPhaseId, clientId, content, attachments[], createdAt	N–1 ProjectPhase
Invoice	id, projectId, milestoneId?, changeRequestId?, amount, amountPaid, status (UNPAID PARTIALLY_PAID PAID OVERDUE), dueDate, sentAt	N–1 Project; born from a milestone OR a change request; 1–N Payments
Payment	id, invoiceId, amount, paidAt, method	N–1 Invoice (manual record; sums into Invoice.amountPaid)
ChangeRequest	id, projectId, type (SCOPE OVERTIME), description, amount, status (REQUESTED APPROVED REJECTED INVOICED), createdBy, approvedBy	N–1 Project; 1–1 Invoice when approved
TransitionLog	id, tenantId, entityType (LEAD PHASE), entityId, fromState, toState, at, byUserId	Append-only; feeds deferred velocity/bottleneck analytics
Notification	id, userId, type, entityId, read, createdAt	N–1 User

9.1 Key Modeling Notes

- Service ↔ Blueprint is many-to-many, derived through phases: a blueprint's service set is the union of its phases' serviceIds.
- The exact-match filter at quotation time compares the selected service set against each blueprint's derived service set for set-equality.
- A phase references exactly one Service; a Service can be referenced by many phases (across one or multiple blueprints).
- Billing is decoupled from phases: money lives on QuotationMilestone, not on individual phases (no PhaseValue entity).
- One-time payment is modeled as a single QuotationMilestone containing all phases, with triggerType ON_PROJECT_START or ON_COMPLETION.
- Each phase belongs to exactly one milestone (enforced via MilestonePhase); milestone amounts must sum to the quotation total.

- An Invoice is always born from a QuotationMilestone, which knows its phases — invoices are never tied directly to a single phase.
- Blueprint immutability for running projects is achieved by snapshot-on-Start: ProjectPhase rows are a full copy of the blueprint's phases (name, service, board, dependencies) taken when the admin clicks Start. Project.blueprintId is retained only as an origin reference, never read for live structure.
- Phase deadlines live on ProjectPhase (not on the blueprint Phase). An optional duration may be set on the blueprint phase; it is materialized into a concrete ProjectPhase.deadline at Start, when the project has a real start date. Overtime alerts compare against ProjectPhase.deadline.
- An Invoice has exactly one source: either a milestoneId or a changeRequestId (never both, never neither). This extends the milestone billing model to cover scope changes and overtime without a separate invoice type.
- Partial payments: Invoice.amountPaid is the sum of its Payment rows; status is derived (UNPAID / PARTIALLY_PAID / PAID / OVERDUE), never set directly. Full gateway reconciliation is deferred — payments are recorded manually in v1.
- A Task always carries exactly one projectPhaseId. There is no task-to-many-phases relation; a shared board is a query across phases, not a shared owner. Phase progress is computed from that phase's tasks only.
- TransitionLog is append-only and written on every lead-stage and phase-status change from day one. No analytics UI reads it in v1, but it is the irreplaceable history that later velocity, completion-time, and bottleneck metrics depend on.
- Authorization is resolved per request: a user's effective permission set = union of permissions across their PermissionGroups, then apply any UserPermissionOverride (GRANT adds, REVOKE removes). Guards check for a specific permission key. There is no role column anywhere in the tenant model — PlatformUser (platform_owner) and Client are the only non-RBAC identities, and both sit outside tenant permissions by design.

10. API Overview

All endpoints are prefixed with `/api/v1` and require a valid JWT Bearer token unless marked public. Every request is validated by Zod schemas. Multi-tenancy is enforced via `@UseGuards()`. Tenant endpoints declare a Required Permission (a permission key) rather than a role; the guard checks the caller's effective permissions. Platform Owner and Client endpoints sit outside tenant RBAC.

Module	Endpoints (summary)	Required permission / actor
Auth	POST <code>/auth/login</code> , POST <code>/auth/refresh</code> , POST <code>/auth/logout</code>	Public
Client onboarding	POST <code>/auth/onboarding/send</code> , GET <code>/auth/onboarding/verify/:token</code> (7-day single-use)	Public (client)
Client login	POST <code>/auth/client-login/send</code> , GET <code>/auth/client-login/verify/:token</code> (15–30 min single-use), POST <code>/auth/client/set-password</code>	Public / Client
Tenants	POST <code>/tenants</code> , GET <code>/tenants/:id</code> , PATCH <code>/tenants/:id/suspend</code>	Platform Owner
Plans (platform)	GET POST PATCH DELETE <code>/platform/plans</code> , manage entitlements	Platform Owner
Permission groups	GET <code>/permission-groups</code> , POST <code>/permission-groups</code> , PATCH <code>/permission-groups/:id</code> , DELETE <code>/permission-groups/:id</code>	permission-groups.manage
Permissions	GET <code>/permissions</code> (catalog)	permission-groups.manage
User groups	POST <code>/users/:id/groups</code> , DELETE <code>/users/:id/groups/:groupId</code>	users.manage
Users	GET <code>/users</code> , POST <code>/users</code> (invite), PATCH <code>/users/:id</code> , DELETE <code>/users/:id</code>	users.view / users.invite / users.manage
Sales flow	GET POST PATCH DELETE <code>/sales-flow/stages</code>	crm.edit
Contacts	GET POST PATCH <code>/contacts</code> , GET <code>/contacts/:id/history</code>	contacts.view / contacts.manage
Leads	GET POST PATCH <code>/leads</code> , PATCH <code>/leads/:id/stage</code> , PATCH <code>/leads/:id/convert</code>	crm.view / crm.create / crm.edit

Services	GET POST PATCH DELETE /services, GET /services/:id/usage	services.view / services.manage
Packages	GET POST PATCH DELETE /packages	packages.manage
Blueprints	GET POST PATCH DELETE /blueprints, GET /blueprints/match?services=	blueprints.view / blueprints.manage
Phases (blueprint)	GET POST PATCH DELETE /blueprints/:id/phases	blueprints.manage
Task boards	GET POST PATCH /task-boards, POST /task-boards/:id/assign-phase	blueprints.manage
Quotations	GET POST PATCH /quotations, POST /quotations/:id/send-proposal	quotations.view / quotations.create / quotations.send
Milestones	GET POST PATCH DELETE /quotations/:id/milestones	quotations.create
Proposals	GET /proposals, POST /proposals/:id/amend, POST /proposals/:id/accept reject	quotations.view (tenant) / Client (portal)
Projects	GET POST /projects, GET /projects/:id/progress, POST /projects/:id/start	projects.view / projects.start
Project phases	GET PATCH /project-phases/:id, POST /project-phases/:id/complete	projects.view / approvals.review
Tasks	GET POST PATCH DELETE /tasks	tasks.view / tasks.manage / tasks.assign
Assets	POST /assets/upload, GET /assets/:id/presigned	tasks.manage (tenant) / Client (portal)
Approvals	POST /approvals, GET /approvals/audit-log	approvals.review (immutable write)
Client feedback	POST GET /feedback	Client (portal)
Invoices	GET POST /invoices, POST /invoices/:id/payments	invoices.view / invoices.create / invoices.record_payment
Change requests	GET POST /projects/:id/change-requests, POST /change-requests/:id/approve reject	change_requests.manage / change_requests.approve (client may POST request)
Analytics	GET /analytics/sales, GET /analytics/delivery, GET /analytics/finance	analytics.view
Platform analytics	GET /platform/analytics (MRR,	Platform Owner

	tenant health)	
Workload	GET /team/workload	users.view
Notifications	GET /notifications, PATCH /notifications/:id/read	All authenticated (own notifications)
Portal	GET /portal/:token/projects, GET /portal/:token/project/:id, POST /portal/:token/approve	Client (token)

Authorization example — instead of 'Role: Admin', an endpoint declares 'Required Permission: quotations.create'. The guard resolves the caller's effective permissions (union of their Permission Groups, adjusted by any overrides) and allows the request only if the key is present.

10.1 WebSocket Events

Event	Direction	Payload
task:updated	Server → Client	{ taskId, status, updatedBy }
phase:completed	Server → Client	{ projectPhaseId, projectId }
approval:submitted	Server → Client	{ approvalId, decision }
client:feedback	Server → Client	{ feedbackId, projectPhaseId, content }
invoice:generated	Server → Client	{ invoiceId, milestoneId, amount, dueDate }
proposal:amended	Server → Client	{ proposalId, versionNo, comment }
project:started	Server → Client	{ projectId, startedAt }
notification:new	Server → Client	{ notificationId, type, message }

11. Service Types & Extensibility

Dalil is domain-agnostic by design. The Blueprint system allows users with blueprints.manage to define any project structure via configuration. The platform ships with templates for the following service types, but any domain with a phased delivery model can be onboarded without code changes.

Service Type	Typical Phases	Notes
Brand Identity	Discovery → Concept → Refinement → Final Delivery	S3 assets critical; fonts, color palettes
Web Design & Dev	Wireframe → Design → Development → QA → Launch	Multi-phase; parallel phases possible
Packaging Design	Concept → 3D Mockup → Print-Ready Files	High-res file delivery
Social Media	Content Plan → Creatives → Scheduling	Recurring monthly deliverables
Print Design	Layout → Proof → Print-Ready	Print spec validation
Marketing Campaign	Strategy → Creative → Launch → Report	KPI tracking per phase
Custom domains	Custom phases (blueprints.manage)	Any service that fits phased delivery

11.1 Blueprint Configuration Rules

- Services must exist before a blueprint can be built.
- A blueprint must have at least one phase.
- Each phase has a name and delivers exactly one service; a service can be delivered across multiple phases.
- A blueprint's service set is the union of its phases' services, used for exact-match filtering at quotation time.
- Phases can be configured as sequential (phase B depends on phase A) or parallel (phases B and C run simultaneously).
- Each phase is assigned a board: a new dedicated board or a board shared with any other phase(s), independent of service.
- Task templates are injected into board columns on project initialization.
- On project Start, the blueprint's phases are snapshotted into the project (copied into ProjectPhase rows). Editing the blueprint afterward does not affect already-started projects — this is how versioning is achieved without separate version entities.
- A blueprint can be reused across any number of projects and clients without modification.

11.2 Exact-Match Filtering — Known Tradeoff (Under Validation)

The exact-match rule (a blueprint's service set must exactly equal the selected services — no subset, no superset) keeps delivery structure unambiguous but has a known scaling cost: an agency selling {Brand}, {Web}, and {Brand+Web} needs three separate blueprints, and each new service combination needs its own blueprint. The 'No blueprint found — create one' fallback prevents dead-ends but could turn quotation into a frequent blueprint-authoring detour.

Decision for v1: retain strict exact-match as the working assumption, on the expectation that a real agency sells a curated handful of service combinations rather than every mathematical permutation. This assumption is flagged for validation against a design partner's actual service matrix before it is locked (see Section 16, open item E). Candidate alternatives if validation shows explosion: (a) allow a superset blueprint with unused phases dropped at Start; (b) compose a quotation from multiple single-service blueprints. Neither is adopted now to avoid over-engineering ahead of real usage data.

12. Non-Functional Requirements

Category	Requirement	Target
Performance	API p95 response time	< 200ms for read endpoints
Performance	Real-time event latency (Socket.io)	< 500ms end-to-end
Scalability	Multi-tenant isolation	Zero data bleed between tenants
Scalability	BullMQ concurrency	Configurable workers; default 5 per queue
Availability	Uptime SLA	99.5% monthly
Security	Audit logs	Immutable; append-only; no delete API
Security	File access	Presigned S3 URLs; no direct bucket exposure
Security	Auth	JWT RS256; refresh token rotation; client onboarding link 7-day single-use; client login link 15–30 min single-use; optional client password (bcrypt/argon2)
i18n	Languages at launch	Arabic (RTL) + English (LTR)
i18n	Font families	Saudi Air (EN — confirm licensed, web-deliverable; see open item F), IBM Plex Sans Arabic (AR — open source)
Responsiveness	Device support	Mobile-first; responsive across

		all screen sizes
Testing	E2E coverage	Critical paths covered by Playwright
Testing	Unit coverage	Target >80% on service layer
Error handling	Error screens	404, 403, 500 Global Error Boundary

13. Subscription & Pricing Model

Dalil operates on a multi-tenant SaaS subscription model. Each tenant (agency) subscribes to a plan. The platform itself is not customer-facing for end clients — the agency's clients access only the white-labelled portal. Plans, pricing, and feature entitlements are owned and managed by the Platform Owner (see 13.1) and are not editable by tenants.

Plan	Price	Limits & Features
Free	SAR 0	1 active project, 1 user, 2 GB storage, basic CRM, client portal
Growth	SAR 249/mo	Up to 10 users, unlimited projects, CRM, proposals, client portal, invoicing, automations
Agency	SAR 599/mo	Unlimited users, white-label, custom domain, API access, advanced reporting
Enterprise	Custom	SSO, dedicated support, custom integrations, SLA, private deployment options

All paid plans include the full delivery workflow (services catalog, blueprints, task boards, milestone billing, real-time notifications). Custom Permission Groups and the full RBAC model are included in every plan, including Free — authorization flexibility is not gated behind Enterprise; only operational limits (users, projects, storage) and the white-label/custom-domain/API features differ by tier. The Free tier is a single-project entry point designed to convert agencies once they run more than one engagement. White-label and custom domain are gated to the Agency tier and above, aligning with the post-MVP custom-domain rollout. Pricing in SAR reflects the primary KSA market; to be validated with early tenants before launch.

Definition of an 'active project' for plan limits: a project counts against the limit while its status is anything other than Completed, Archived, or Cancelled. Reaching the final completed/approved state frees the slot, so a Free-tier tenant can start a new project once their previous one is completed. A project in the 'not yet started' holding state (accepted proposal, awaiting admin Start) does not yet count as active.

13.1 Platform Administration (Platform Owner)

Above all tenants sits the Platform Owner — the company operating Dalil. This layer manages the SaaS business and its commercial rules. It has no access to any tenant's internal operational data (projects, clients, deliverables, messages); it controls only the container and the commercial

configuration. This separation is what allows plans and pricing to be updated centrally without ever touching tenant workspaces.

Platform Owner Capabilities

- Manage subscription plans: create, edit, rename, retire plans; adjust pricing (e.g. update the SAR amounts) without code changes.
- Manage feature entitlements: define which features each plan unlocks (e.g. white-label, custom domain, API access, advanced reporting) via a feature-flag / entitlement map.
- Tenant lifecycle: provision new tenants, suspend or reactivate accounts, view tenant subscription status and platform-level billing.
- Platform settings: global configuration, announcements, system health and usage metrics across all tenants (aggregate, not content).
- Explicitly out of scope: viewing or editing a tenant's projects, clients, proposals, deliverables, or any operational content.

Entitlement Model

Features are gated by a plan-to-entitlement map rather than hard-coded per plan. Each plan holds a set of entitlement flags (e.g. `white_label`, `custom_domain`, `api_access`, `advanced_reporting`, `max_users`, `max_projects`, `storage_gb`). The application checks entitlements at runtime, so the Platform Owner can change what a plan includes — or introduce a new plan — purely through configuration. This keeps the monetization strategy fully adjustable post-launch.

14. DevOps & Infrastructure

Area	Tool / Approach
CI/CD	GitHub Actions: lint → type-check → test → build → deploy on merge to main
Git hooks (pre-commit)	Husky + lint-staged: run ESLint, Prettier, and TypeScript type-check on staged files; block commit on any error
Git hooks (pre-push)	Husky pre-push: run full test suite and production build; block push if tests fail or build errors
Commit message linting	commitlint with Conventional Commits: enforce type(scope): subject format, subject length, and body/footer formatting via Husky commit-msg hook
Development environment	Devcontainer (consistent across team; Docker-based)
Version control	GitHub; feature branches + PR review required;

	product-scoped repo naming (dalil-api, dalil-web) to keep room for future buslahub sibling products
Environment config	Environment variables via .env; secrets in GitHub Actions secrets
Database migrations	Prisma migrate; run in CI before deploy
File storage	AWS S3 (me-south-1 / me-central-1) with per-tenant path prefix; lifecycle rules for old assets
Monitoring	TBD — recommended: Sentry (errors) + Datadog or BetterStack (logs/metrics)
Deployment target	TBD — recommended: AWS ECS / Railway / Render for MVP

14.1 Pre-Commit & Commit-Message Quality Gates

To keep the main branch always green and the history readable, the repository enforces local quality gates via Husky before code ever reaches CI:

- Pre-commit hook: lint-staged runs ESLint (fix), Prettier (format), and tsc --noEmit type-check on staged files. Any lint error, type error, or formatting violation blocks the commit.
- Pre-push hook: the full Jest unit suite and a production build (next build / nest build) must pass with zero errors before the push is allowed.
- Commit-msg hook: commitlint validates the message against the Conventional Commits standard.
 - Required format: type(scope): subject — e.g. feat(quotations): add milestone grouping step.
 - Allowed types: feat, fix, docs, style, refactor, perf, test, chore, ci, build.
 - Subject must be ≤72 characters, lower-case, imperative mood, no trailing period.
 - Body (optional) is wrapped at 100 characters per line and separated from the subject by a blank line; footer carries breaking-change notes and issue references.

These gates ensure that TypeScript compiles, linting passes, tests are green, the build succeeds, and every commit message is consistently formatted — enforced on every developer machine, not just in CI.

15. Phased Development Roadmap

Phase	Scope	Duration (est.)
Phase 0 — Foundation	Auth (JWT + magic link), multi-tenancy, platform-owner layer + plan/entitlement model, RBAC (permission groups, permissions, tenant bootstrap, admin protection rules), base DB schema, CI/CD + Husky git hooks	4 weeks
Phase 1 — Catalog & CRM	Services catalog, packages, custom sales flow configurator, lead pipeline board, contact profile, conversion + magic link flow	5 weeks
Phase 2 — Blueprint Engine	Blueprint CRUD, phases (one service per phase), phase dependencies (sequential/parallel), task board assignment (dedicated/shared). Built before Quotation because the quotation's exact-match filter and milestone grouping operate on blueprints and phases.	4 weeks
Phase 3 — Quotation & Proposal	Quotation builder (service/package selection, exact-match blueprint filter, total computation), milestone grouping over phases, all payment models, proposal generator, amendment loop, accept/start flow with blueprint snapshot	4 weeks
Phase 4 — Task Board	Kanban board, task CRUD, shared boards, file upload, real-time updates via Socket.io	4 weeks
Phase 5 — Approval & Client Portal	Approval queue (approvals.review), client portal (all screens), feedback loop, amendment flow, audit log	5 weeks
Phase 6 — Invoicing, Payments & Change Requests	BullMQ automations for all payment models, invoice generation, email + portal delivery, partial payments (amountPaid + Payment rows), change requests + additional invoices, overtime-as-change-	4 weeks

	request	
Phase 7 — Polish, Analytics & i18n	v1 analytics dashboards (tenant + platform), workload view, Arabic RTL support, responsive design audit, error screens, Playwright tests. Transition-timestamp logging is wired in from Phase 1 onward, not here.	4 weeks
Phase 8 — Beta & Launch	Self-serve onboarding flow, billing integration (Stripe), documentation, design-partner validation of exact-match rule	4 weeks

Total estimated build time: ~36 weeks (9 months) with a team of 2–3 full-stack engineers + 1 designer. Timeline can be compressed with parallel workstreams.

16. Resolved Decisions & Open Items

The following key decisions have been resolved during planning. They are recorded here and reflected throughout the relevant sections of this document.

16.1 Resolved Decisions

#	Decision	Resolution
1	Product name	Named Dalil, delivered under the buslahub.com umbrella brand at dalil.buslahub.com. Final trademark check still advisable before public launch
2	Client portal white-label / custom domain	Subdomain per tenant at launch; custom domains gated to Agency tier and above, delivered post-MVP
3	Payment gateway	Stripe for tenant subscriptions; verify Stripe KSA/SAR support for client invoicing, with Moyasar/Tap/HyperPay as local fallback
4	E-signature	Not at launch; authenticated 'Accept' action + immutable audit log serves as acceptance record. E-signature is post-MVP

5	Data residency	Launch in AWS Middle East region (me-south-1 Bahrain or me-central-1 UAE) for KSA PDPL alignment and latency; design region-portable
6	Mobile app vs web	Responsive web only at launch (mobile-first Next.js); native app deferred until post product-market fit
7	Magic link TTL	7 days single-use for account-creation/onboarding; 15–30 min single-use for any login-style links
8	Blueprint templates at launch	3–5 polished reference blueprints across core service types; users duplicate and customise
9	Onboarding flow	Assisted/manual onboarding for early tenants; self-serve signup built in a later phase
10	Overtime / additional charges	Manually entered line items (change_requests.manage) at launch; auto-calculation deferred
11	Client in multiple tenants	Schema supports one person → many tenant portals from day one (global identity by email, tenant-scoped memberships); minimal UI surface
12	Milestone grouping rule	Milestones must be dependency-closed (a phase can't be grouped later than something that depends on it), which generalizes contiguity to parallel phases. Milestone firing respects milestone-level dependency order.
13	Editing/deleting a referenced service	Service details always editable; deletion blocked when referenced — archive instead; running project references never broken
14	Package discounts	Fixed bundled price in v1; percentage-based discounts deferred

16.2 Remaining Open Items

A small number of items remain to be confirmed, primarily external verifications rather than design decisions:

#	Item	Owner / Next Step
A	Confirm Stripe payout support for Saudi bank accounts in SAR; finalise local gateway if needed	Finance / founder — verify with Stripe + local provider
B	Trademark check for 'Dalil' and the buslahub.com umbrella brand before public launch	Founder — before any public-facing branding
C	Select monitoring stack (Sentry + Datadog/BetterStack) and deployment target (ECS / Railway / Render)	Engineering — during Phase 0
D	Validate SAR pricing tiers with first cohort of early tenants	Founder — during assisted onboarding
E	Pressure-test the strict exact-match blueprint rule against a real agency's service matrix; adopt a fallback (superset-with-drop or multi-blueprint composition) only if explosion is confirmed	Product — with first design partner, before locking the model
F	Confirm 'Saudi Air' is a licensed, web-deliverable typeface (not a placeholder); secure web font licence or pick an alternative	Design / founder — before Phase 7 (i18n & polish)

All twelve items from the internal design review (v1.3) have been resolved in this version: roadmap sequencing (blueprint before quotation), client re-entry login, the milestone parallel-phase rule, exact-match validation flag, quotation total computation, blueprint snapshot-on-Start, phase deadlines, MRR currency, proposal reject/reopen states, persona wording, free-tier active-project definition, and the font licensing flag.

17. Glossary

Term	Definition
Tenant	An agency or freelancer subscribing to Dalil; the top-level isolation unit.
Tenant User	Any authenticated person inside a tenant. Has no fixed role; all access comes from membership in one or more Permission Groups.
RBAC	Role-Based Access Control — Dalil's authorization model where permissions (not roles) are the source of truth, bundled into Permission Groups and assigned to users.
Permission	An atomic authorization key in the form <code>category.action</code> (e.g. <code>quotations.create</code>). Enforcement checks for a specific permission, never a role name.
Permission Group	A tenant-scoped, named bundle of permissions assigned to users. Default groups (Administrator, Sales, Finance, ...) ship as customizable starting templates.
Effective Permissions	A user's resolved permission set: the union of all permissions across their Permission Groups, adjusted by any per-user overrides.
Administrator (group)	The system Permission Group holding all tenant permissions. Cannot be deleted; a tenant must always retain at least one member (system invariant).
Platform Owner	The company operating Dalil, sitting above all tenants. Manages plans, pricing, entitlements, and tenant lifecycle — but never a tenant's operational data.
Plan	A subscription tier (Free, Growth, Agency, Enterprise) with a price and a set of feature entitlements, managed by the Platform Owner.
Entitlement	A feature flag or limit attached to a plan (e.g. <code>white_label</code> , <code>api_access</code> , <code>max_users</code>) that the app checks at runtime to gate features.
Service	A sellable offering with a name, description, and default price. The atomic commercial unit; each phase in a blueprint delivers exactly one service.
Package	A commercial bundle of multiple services sold at a combined price; expands to its underlying service set at quotation time.
Service Set	The set of services a blueprint covers, derived

	from the union of its phases' services. Used for exact-match filtering.
Exact-Match Filter	At quotation time, only blueprints whose service set exactly equals the selected services are shown — no subset, no superset.
Custom Sales Flow	A tenant-defined set of pipeline stages that leads progress through; fully configurable by a user with <code>crm.edit</code> (number, names, order).
Blueprint	A reusable project structure template defining phases, dependencies, and task board assignments. Selected at quotation time and reusable across unlimited projects.
Phase	A discrete stage of a project with its own name, delivering exactly one service, with a task board and dependency config.
Phase Dependency	A rule configured in the blueprint: sequential (phase B waits for phase A) or parallel (phases run simultaneously).
Task Board	A kanban board assigned to one or more phases in a blueprint. Can be dedicated (one phase) or shared (multiple phases, any service).
Milestone	A billing checkpoint grouping one or more phases (dependency-closed). Triggers a single invoice when all its phases are completed and approved, respecting milestone-level firing order.
Milestone Grouping	The quotation-time action of drawing billing boundaries across a blueprint's phases to define milestones.
Payment Model	The invoicing strategy: one-time (single all-phase milestone, before start or after finish) or milestone-based (multiple milestones).
Onboarding Link	A one-time, 7-day URL sent to a converted lead to create their account and provision their portal. Used once, at conversion only — not for subsequent logins.
Client Login Link	A short-TTL (15–30 min), single-use passwordless URL a returning client requests by email to sign back into their portal. The primary day-to-day client login path.
Dependency-Closed Milestone	A milestone whose phases include every phase lying on a dependency path between any two of its members — the rule that replaces simple contiguity once phases can run in parallel.
Snapshot-on-Start	Copying a blueprint's phases into ProjectPhase rows when the admin clicks Start, making the project independent of later blueprint edits (the

	versioning mechanism).
Change Request	An admin-controlled scope or overtime adjustment on a project that, once approved, generates a standalone additional invoice outside the milestone structure.
Partial Payment	A recorded payment covering part of an invoice; Invoice.amountPaid sums all payments and the status (Unpaid/Partially paid/Paid/Overdue) is derived from it.
Transition Log	An append-only record of every lead-stage and phase-status change, captured from day one as the data foundation for later velocity and bottleneck analytics.
Workload	The count and list of active tasks assigned to a team member across all projects — the v1 team-management view (capacity planning and time tracking are deferred).
Concept	A creative deliverable card on the task board representing a proposal to the client.
Portal	The white-labelled web interface accessible only to the client for their projects.
Amendment Loop	The iterative cycle of client commenting on a proposal, admin revising, and resending until the client accepts.
Audit Log	An immutable, append-only record of every milestone approval and rejection action.
BullMQ	The Redis-backed async job queue used for invoice triggers, emails, and automation.
Presigned URL	A time-limited AWS S3 URL for secure file upload/download without exposing the bucket.

End of Document — Dalil PRD v1.6